

Songs Discovery Service with Cassandra Vector DB



Estimated time needed: **30** minutes

Objectives

After completing this lab, you will be able to:

- Generate vector embeddings for the data of song titles using AI models
- Access the Cassandra database and create a table with Vector Embeddings
- Develop an endpoint using Express server that accepts the sentence as input and provides the nearest song title in response

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) is an open-source IDE (Integrated Development Environment) that provides an environment for hands-on labs in course and project-related labs.

Important notice about the lab environment

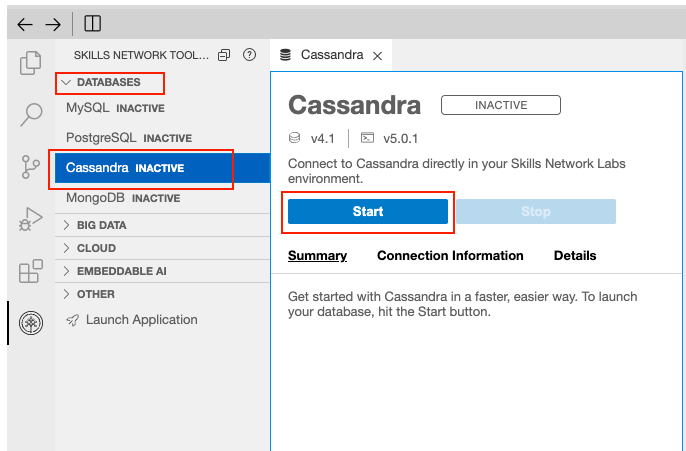
Please be aware that sessions for this lab environment are not persistent. Each time you connect to this lab, it creates a new environment for you. You will lose any data saved in the earlier session. Plan to complete these labs in a single session to avoid losing your data.

Task 1: Prerequisites

Start the Cassandra database

Cassandra database is provided in the lab environment. Follow the steps in this task to start the database.

1. Open the Database section on the left and click on Cassandra. You will see Cassandra appear in the right panel. Click on **Start** to start the Cassandra database.



Once the database starts, note down the username and password as you will need them later.

SKILLS NETWORK TOOLBOX

▼ DATABASES

MySQL INACTIVE

PostgreSQL INACTIVE

Cassandra **ACTIVE**

MongoDB INACTIVE

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Application

Cassandra

ACTIVE

v4.1 | v5.0.1

Connect to Cassandra directly in your Skills Network Labs environment.

Start Stop

Summary Connection Information Details

Your database server is now ready to use and available with the following login credentials. For more details on how to navigate Cassandra, please check out the Details section.

Username: captainfedo1

Password: NDA5LWNhcHRhaW5m

To interact with the database in the terminal, select one of these options:

Cassandra CLI New Terminal

Set up the directory structure

1. Open a terminal in the lab environment using the Terminal -> New Terminal menu.

File Edit Selection View Go Run **Terminal** Help

← →

SKILLS NETWORK TOOLBOX

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Application

New Terminal

New Terminal (With Profile)...

Choose Default Profile...

Run Task...

Run Build Task

Run Test Task

Rerun Last Task ⌘ K

Show Running Tasks...

Restart Running Task...

Terminate Task...

Attach Task...

Configure Tasks...

2. Next, create the home directory. Let's call this cassandra-songs.

```
mkdir cassandra-songs && cd cassandra-songs
```

Let's also save the full path of the folder in an environment variable that you can refer to later.

```
export VECTOR_HOME=$(pwd)
```

You can use the echo command to see what's inside this variable.

```
echo $VECTOR_HOME
```

The output appears as follows. It will look different on your computer.

```
echo $VECTOR_HOME  
/home/project/cassandra-songs
```

Let's create a data sub-directory using this variable.

```
mkdir $VECTOR_HOME/data
```

You should see a data directory inside the project directory.

```
ls -ltr  
total 4  
drwxr-sr-x 2 theia users 4096 Apr 18 18:00 data
```

You will use the cosine function to search for similar news items. The other functions available are:

- cosine: Measures the cosine of the angle between two vectors. A higher cosine value indicates more remarkable similarity.
- euclidean: Measures the "straight-line" distance between two points (vectors) in multi-dimensional space. A lower distance indicates higher similarity.
- dotproduct: Calculates the sum of the product of corresponding components between two vectors. Higher dot product values indicate greater alignment.

[Learn more in the Cassandra documentation](#)

Task 2: Seed Data

Before writing any code, let's look at the data file quickly. The seed data provided is a CSV file containing an array of songs. Each song in the file has the following attributes:

- Artist: Name of the artist
- Title: Title of the song
- Year: Year of song release
- Sales: Total sales for the song
- Streams: Number of times the song was streamed
- Downloads: Number of downloads for the song
- Radio plays: Number of times the song was played on the radio
- Rating: Numerical rating

Artist	Title	Year	Sales
Urban Echoes	City Lights Serenade: The pulse of neon in the urban night. Stories told through the whispers of the wind between skyscrapers.	2021	16.300
Nature's Melody	Forest Whispers and Waterfall Murmurs: Every leaf, a symphony; every ripple, a song. Nature's orchestra plays the eternal tunes of the wild.	2019	18.750
Digital Symphony	Binary Beats: A journey through the circuits and bytes of rhythm. Where technology meets melody, creating the soundtrack of the digital age.	2020	20.987
Timeless Classics	Through the Ages: Echoes of time resonating in every note. Rediscover the classics reinterpreted for today's ears.	2018	15.432
Global Anthems	Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.	2017	22.500
Jazz Chronicles	Jazz from the Alleys: Stories of night, tales of light. The saxophone narrates, the trumpet debates, and the rhythm captivates.	2019	13.200
Heartland Harmonies	Prairie Songs: Sung under open skies, carried by the breeze. Tales of heartland, sung with soul, echoing the spirit of the plains.	2020	14.200
Rock Revolution	Guitar Legends: Strings that screamed revolutions, drums that dictated new eras. A collection where each chord tells a story of rebellion and freedom.	2018	21.764
Morning Acoustics	Sunrise Melodies: As the day begins, so does the tune. Gentle strums that accompany the first light, bringing warmth to the waking world.	2021	19.900
Dance Illuminations	Dance of Lights: Beats that make the shadows dance. Music that turns the night into an illuminated stage for all.	2022	17.567
Country Reflections	Songs of the Soil: Stories woven from the threads of farm life. Each lyric a reflection of love, labor, and land.	2018	12.870
Island Rhythms	Island Heartbeat: Feel the rhythm of the sands, the melody of the ocean. Music that carries the soul to the shores of tranquility.	2017	15.634
Operatic Echoes	Voices of the Opera: Power and passion collide in soaring melodies. Experience the dramatic tales as told by the masters of vocal art.	2021	18.980
Disco Nights	Disco Days, Disco Nights: When the lights spun as fast as the records. Dive back into the era where the beats dictated the pace of the heartbeat.	2019	16.500
Punk Power	Punk Spirit: Not just music, but a movement. The raw energy and the untamed melodies that shaped a generation ready to stand.	2018	15.790
Heavy Echoes	Metal Titans: Heavy riffs and thunderous drums echo through the halls of rock. Relive the power that forged the metal legacy.	2019	18.432
Symphonic Stories	Epic Orchestras: Tales told in the language of strings and horns. Each composition a chapter, every note a word in the saga of sound.	2020	21.768
Urban Poetry	Rhymes of the Street: The beat of the concrete, the rhythm of the city. Hear the stories of the streets narrated by the voices of the poets.	2022	23.345
Jazz Soothe	Evening Jazz: Let the troubles of the day melt away with each note. A smooth transition from day to night through the calming essence of jazz.	2017	18.567
Soulful Nights	Nights of Soul: Where the heart sings the blues, and the soul feels the beats. Dive deep into the emotional landscapes painted by soulful melodies.	2018	20.432

Data attribution

“[Top Songs of the World](#)” by [Kaggle](#) is licensed under [CC0 1.0 DEED](#). The titles in the original data were extended to illustrate vector embeddings and similarity search better.

You can get the complete file using the following `wget` or `curl` command or open this [URL](#) in your browser and save the file locally.

Use the following commands in the lab environment.

WGET

```
wget -O $VECTOR_HOME/data/songs.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/BTCu1CIn1cpNi9gAU-h2jQ/songs.csv
```

CURL

```
curl -o $VECTOR_HOME/data/songs.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/BTCu1CIn1cpNi9gAU-h2jQ/songs.csv
```

Task 3: Create Node.js project and install dependencies

Let's create the node package and install all the dependencies.

1. Ensure you are still in the `$VECTOR_HOME` directory and run the `npm init -y` command.

```
npm init -y
```

The output should look something like this:

```
→ cassandra-songs npm init -y
Wrote to /home/project/cassandra-songs/package.json:
{
  "name": "cassandra-songs",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

2. Install required packages including `@tensorflow-models/universal-sentence-encoder`, `@tensorflow/tfjs-node`, `dotenv`, `cassandra-driver`, `csv` parser, and `express`.

- `@tensorflow-models/universal-sentence-encoder`: The universal sentence encoder model
- `@tensorflow/tfjs-node`: JavaScript version of TensorFlow

- dotenv: Loads environment variables from a .env file into process.env
- cassandra-driver: Library to work with Cassandra DB
- csv-parser: To read and parse csv file
- express: Web framework for Node.js

```
npm install @tensorflow-models/universal-sentence-encoder @tensorflow/tfjs-node dotenv express cassandra-driver csv-parser
```

The output is long but should end with libraries installed:

```
added 189 packages, and audited 190 packages in 31s
21 packages are looking for funding
  run `npm fund` for details
2 moderate severity vulnerabilities
To address all issues, run:
  npm audit fix
Run `npm audit` for details.
```

You can safely ignore the warnings for now.

Task 4: Create common util functions

Create a util.js file that has the common reusable code

1. Create a new file name util.js.

```
touch util.js
```

2. Open the file in the editor and paste the following content.

Please read the code comments to understand what each part of the code does.

```
require('dotenv').config();
const cassandra = require('cassandra-driver');
const TimeUuid = cassandra.types.TimeUuid;
require("@tensorflow/tfjs-node");
const us_encoder = require("@tensorflow-models/universal-sentence-encoder");
let client;
let model;
// function to connect to the database and return a client.
async function connectToCassandra() {
  client = new cassandra.Client({
    contactPoints: [process.env.CASSANDRA_HOST],
    port: process.env.CASSANDRA_PORT,
    localDataCenter: 'datacenter1',
    authProvider: new cassandra.auth.PlainTextAuthProvider(process.env.CASSANDRA_USER, process.env.CASSANDRA_PASS)
  });
  await client.connect();
  console.log('Connected to Cassandra');
  return client;
}
// method that ensures there is a keyspace and table created in the database.
async function ensureKeyspaceAndTable() {
  console.log('Ensuring KEYSpace ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} exists ...');
  const keyspaceQuery = `CREATE KEYSpace IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE} WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};`;
  await client.execute(keyspaceQuery);
  const tableQuery = `CREATE TABLE IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE} {`
```

```

    id uuid PRIMARY KEY,
    artist text,
    title text,
    year int,
    sales float,
    streams float,
    downloads float,
    radio_plays float,
    rating float,
    embedding VECTOR <FLOAT, 512>
);
await client.execute(tableQuery);
console.log(`KEYSPACE ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} created`);
// Ensure the vector index exists for efficient vector search
const indexQuery = `CREATE INDEX IF NOT EXISTS ann_index ON ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE}(embedding) USING 'sai';`;
await client.execute(indexQuery);
console.log('index created');
}
// we cache the model so we don't have to fetch it every time
async function loadModel() {
  if (!model) {
    model = await us_encoder.load();
  }
}
async function vectorizeText(text) {
  await loadModel();
  const embeddings = await model.embed([text]);
  return embeddings.arraySync()[0]; // return the first embedding
}
async function closeCassandraConnection() {
  if (client) {
    await client.shutdown();
    console.log('Cassandra connection closed');
  }
}
// export all functions and variables so other files can use it.
module.exports = {
  connectToCassandra,
  ensureKeyspaceAndTable,
  vectorizeText,
  closeCassandraConnection,
  TimeUuid,
  CASSANDRA_KEYSPACE: process.env.CASSANDRA_KEYSPACE,
  CASSANDRA_TABLE: process.env.CASSANDRA_TABLE
};

```

Let's look at each function in detail.

Import statements

You import a few libraries at the top of the file:

- `require('dotenv').config();` imports variables from .env file. This file has the `CASSANDRA_HOST`, `CASSANDRA_PORT`, `CASSANDRA_KEYSPACE`, and `CASSANDRA_TABLE` constants defined. You would not normally share this file in the GitHub repo or with anybody else, but you are sharing it here to make the exercise straightforward. Feel free to change the values in this file.
- `const cassandra = require('cassandra-driver');` imports the `cassandra` npm package. This package interacts with Cassandra DB from a JavaScript program.
- `require("@tensorflow/tfjs-node");` used in Node.js applications to include TensorFlow.js for Node.js, a JavaScript library for training and deploying machine learning models in Node.js.

Connect to Cassandra

You first create a client to connect with Cassandra. This client must know where the database runs (`CASSANDRA_HOST`) and the port (`CASSANDRA_PORT`). You can use the `connect` method to make the connection.

```

async function connectToCassandra() {
  client = new cassandra.Client({
    contactPoints: [process.env.CASSANDRA_HOST],
    port: process.env.CASSANDRA_PORT,
    localDataCenter: 'datacenter1',
    authProvider: new cassandra.auth.PlainTextAuthProvider(process.env.CASSANDRA_USER, process.env.CASSANDRA_PASS)
  });
  await client.connect();
  console.log('Connected to Cassandra');
  return client;
}

```

Ensure keyspace and table exist

The keyspace is the top-level container that encapsulates database objects like tables (also called column families), types, user-defined functions, and indexes. The method ensures that the keyspace `process.env.CASSANDRA_KEYSPACE` and the table `process.env.CASSANDRA_TABLE` is available in the database. Note the different columns and their types that correspond to the CSV file. In particular, the `embedding` column is of the type `VECTOR` and contains 512 floats.

```
async function ensureKeyspaceAndTable() {
  console.log(`Ensuring KEYSpace ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} exists ...`);
  const keyspaceQuery = `CREATE KEYSpace IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE} WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};`;
  await client.execute(keyspaceQuery);
  const tableQuery = `CREATE TABLE IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE} (
    id timeuuid PRIMARY KEY,
    artist text,
    title text,
    year int,
    sales float,
    streams float,
    downloads float,
    radio_plays float,
    rating float,
    embedding VECTOR <FLOAT, 512>
  );`;
  await client.execute(tableQuery);
  console.log(`KEYSPACE ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} created`);
  // Ensure the vector index exists for efficient vector search
  const indexQuery = `CREATE INDEX IF NOT EXISTS ann_index on ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE}(embedding) USING 'sai';`;
  await client.execute(indexQuery);
  console.log(`index created`);
}
```

Create a vector from a sentence

The `vectorizeText` function takes text as input and creates an embedding or a vector using the `universal-sentence-encoder` model from TensorFlow. You are also using the `loadModel()` method to ensure the model is only downloaded once, thus improving performance.

```
let model;
async function loadModel() {
  if (!model) {
    model = await us_encoder.load();
  }
}
async function vectorizeText(text) {
  await loadModel();
  const embeddingsRequest = await model.embed(text);
  const vector = embeddingsRequest.arraySync()[0];
  return vector;
}
```

Close the database connection

Finally, the `closeCassandraConnection` method closes the connection using the `shutdown` method.

```
async function closeCassandraConnection() {
  if (client) {
    await client.shutdown();
    console.log(`Cassandra connection closed`);
  }
}
```

Finally, the file exports all the functions and variables for use elsewhere.

```
module.exports = {
  connectToCassandra,
  ensureKeyspaceAndTable,
```

```
vectorizeText,
closeCassandraConnection,
TimeUuid,
CASSANDRA_KEYSPACE: process.env.CASSANDRA_KEYSPACE,
CASSANDRA_TABLE: process.env.CASSANDRA_TABLE
};
```

Task 5: Load the vectors into Cassandra DB

Initial data

The system saves the songs in the songs.csv file. This exercise aims to read this file into memory and then create vector embeddings for each title and store them in the database along with the rest of the columns. In the database, this will appear as follows:

```
c3fac660-fdf9-11ee-bc2f-5144b972d0c9 |
Digital Symphony |
11.345 |
[-0.014376, -0.074318, -0.002296,
0.036089, -0.007423, -0.02917, -0.016283,
-0.05468, 0.041119, -0.054654, -0.011597,
... -0.039793, 0.00193, -0.052968] |
10.876 |
4.6 |
20.987 |
18.764 |
Binary Beats: A journey through the circuits and bytes of rhythm. Where technology meets melody, creating the soundtrack of the digital age. |
2020
```

Each row is a column here. The 4th column is embedding, an array of 512 decimal numbers that describe the song's title as a vector. The column will be compared to the incoming text when searching for a song. Here is the table saved in a CSV file for visualization. The `embedding` and `title` columns are highlighted.


```
CASSANDRA_HOST=localhost
CASSANDRA_USER=
CASSANDRA_PASS=
CASSANDRA_PORT=9042
CASSANDRA_KEYSPACE=vks
CASSANDRA_TABLE=vectors
```

Fill the CASSANDRA_USER and the CASSANDRA_PASS entries with the username and password shown when you started the server in the prerequisite step. If you don't have them handy, or the server has stopped, you will need to restart the database. You will get a fresh pair of username and password everytime the database finishes starting.

File to upload vectorized data into the collection

Create a new file named loader.js.

```
touch loader.js
```

The file will contain the method that loads the initial news articles to the Cassandra database.

```
const fs = require('fs');
const csv = require('csv-parser');
const { connectToCassandra, ensureKeyspaceAndTable, vectorizeText, closeCassandraConnection, TimeUuid, CASSANDRA_KEYSPACE, CASSANDRA_TABLE } = require('./util');
async function loadVectorizedData() {
  const client = await connectToCassandra(); // Ensure keyspace and table are set up here
  await ensureKeyspaceAndTable();
  const operations = []; // To store all the async operations
  const dataStream = fs.createReadStream('./data/songs.csv')
    .pipe(csv())
    .on('data', (row) => {
      dataStream.pause();
      const operation = async () => {
        try {
          const sentence = row['Title'];
          console.log(`read sentence: ${sentence}`);
          const embedding = await vectorizeText(sentence);
          const query = `INSERT INTO ${CASSANDRA_KEYSPACE}.${CASSANDRA_TABLE} (id, artist, downloads, embedding, radio_plays, rating, sales, streams, title, year)
VALUES (${TimeUuid.now()}, ${row['Artist']}, ${parseFloat(row['Downloads'])}, ${embedding}, ${parseFloat(row['Radio Plays'])}, ${parseFloat(row['Rating'])}, ${parseFloat(row['Sales'])}, ${parseFloat(row['Streams'])}, ${sentence}, ${parseInt(r
          await client.execute(query);
          console.log(`Inserted sentence: ${sentence}`);
        } catch (error) {
          console.error('Error inserting data:', error);
        } finally {
          dataStream.resume();
        }
      };
      // push the promise to the array; the promise has not been executed yet
      operations.push(operation());
    })
    .on('end', async () => {
      // Wait for all operations to complete before closing the connection
      await Promise.all(operations);
      await closeCassandraConnection();
      console.log('Finished loading data');
    });
  dataStream.read(); // Start reading from the stream
}
loadVectorizedData();
```

The code looks a little complicated, but fear not; you will go through each line as before.

Import initial data and common util functions

First, import the file stream so you can read the CSV file and the CSV parser to parse it. Next, you import all the methods and variables from the util.js file.

```
const fs = require('fs');
const csv = require('csv-parser');
const { connectToCassandra, ensureKeyspaceAndTable, vectorizeText, closeCassandraConnection, TimeUuid, CASSANDRA_KEYSPACE, CASSANDRA_TABLE } = require('./util');
```

Read the file and create embeddings

```
const dataStream = fs.createReadStream('./data/songs.csv')
  .pipe(csv())
  .on('data', (row) => {
```

- The stream is piped through a CSV parser with .pipe(csv()), which processes the CSV row by row.
- Each row triggers a 'data' event: .on('data', (row) => {...}).

Handle the asynchronous nature

```
dataStream.pause();
```

This bit is crucial to ensure the data is available when you create the embedding in the following few lines. This line pauses the data stream so you can make a promise.

```
const operation = async () => {
  try {
    const sentence = row['Title'];
    console.log('read sentence: ${sentence}');
    const embedding = await vectorizeText(sentence);
    const query = `INSERT INTO ${CASSANDRA_KEYSPACE}.${CASSANDRA_TABLE} (id, artist, downloads, embedding, radio_plays, rating, sales, streams, title, year)
VALUES (${TimeUuid.now()}, ${row['Artist']}, ${parseFloat(row['Downloads'])}, [${embedding}], ${parseFloat(row['Radio Plays'])}, ${parseFloat(row['Rating'])}, ${parseFloat(row['Sales'])}, ${parseFloat(row['Streams'])}, '${sentence}', ${parseInt(r
    console.log('Inserted sentence: ${sentence}');
  }
}
```

You create a promise using the vectorizeText method to generate the vector embedding. **It is important to note that you are not executing the promise but creating it.** It might look as if you are inserting documents into the database, but you are just creating the promise to do so.

```
catch (error) {
  console.error('Error inserting data:', error);
} finally {
  dataStream.resume();
}
};
```

Here, you resume the datastream in the `finally` clause to continue reading the file.

```
.on('end', async () => {
  // Wait for all operations to complete before closing the connection
  await Promise.all(operations);
  await closeCassandraConnection();
  console.log('Finished loading data');
});
```

Finally, on the end event, when the datastream has ended or, in other words, you have read the whole file, use the `Promise.all()` method to execute all the promises. These promises are executed in parallel.

Before you run the code, you need to set up the `.env` file, which contains the connection configuration for connecting to Cassandra DB.

Run the code

1. Install the packages
Install all the packages using the `npm install` command.

```
npm install
```

2. Run the file
Run the file using the node command.

```
node loader.js
```

You should see the new articles load one by one:

```
The Platform node has already been set. Overwriting the platform with node.
Connected to Cassandra
Ensuring KEYSPACE vks and TABLE vectors exists ...
KEYSPACE vks and TABLE vectors created
index created
read sentence: City Lights Serenade: The pulse of neon in the urban night. Stories told through the whispers of the wind between skyscrapers.
Inserted sentence: City Lights Serenade: The pulse of neon in the urban night. Stories told through the whispers of the wind between skyscrapers.
read sentence: Forest Whispers and Waterfall Murmurs: Every leaf, a symphony; every ripple, a song. Natures orchestra plays the eternal tunes of the wild.
Inserted sentence: Forest Whispers and Waterfall Murmurs: Every leaf, a symphony; every ripple, a song. Natures orchestra plays the eternal tunes of the wild.
read sentence: Binary Beats: A journey through the circuits and bytes of rhythm. Where technology meets melody, creating the soundtrack of the digital age.
Inserted sentence: Binary Beats: A journey through the circuits and bytes of rhythm. Where technology meets melody, creating the soundtrack of the digital age.
read sentence: Through the Ages: Echoes of time resonating in every note. Rediscover the classics reinterpreted for todays ears.
Inserted sentence: Through the Ages: Echoes of time resonating in every note. Rediscover the classics reinterpreted for todays ears.
read sentence: Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.
Inserted sentence: Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.
read sentence: Jazz from the Alleys: Stories of night, tales of light. The saxophone narrates, the trumpet debates, and the rhythm captivates.
Inserted sentence: Jazz from the Alleys: Stories of night, tales of light. The saxophone narrates, the trumpet debates, and the rhythm captivates.
read sentence: Prairie Songs: Sung under open skies, carried by the breeze. Tales of heartland, sung with soul, echoing the spirit of the plains.
Inserted sentence: Prairie Songs: Sung under open skies, carried by the breeze. Tales of heartland, sung with soul, echoing the spirit of the plains.
read sentence: Guitar Legends: Strings that screamed revolutions, drums that dictated new eras. A collection where each chord tells a story of rebellion and freedom.
Inserted sentence: Guitar Legends: Strings that screamed revolutions, drums that dictated new eras. A collection where each chord tells a story of rebellion and freedom.
read sentence: Sunrise Melodies: As the day begins, so does the tune. Gentle strums that accompany the first light, bringing warmth to the waking world.
Inserted sentence: Sunrise Melodies: As the day begins, so does the tune. Gentle strums that accompany the first light, bringing warmth to the waking world.
read sentence: Dance of Lights: Beats that make the shadows dance. Music that turns the night into an illuminated stage for all.
Inserted sentence: Dance of Lights: Beats that make the shadows dance. Music that turns the night into an illuminated stage for all.
read sentence: Songs of the Soil: Stories woven from the threads of farm life. Each lyric a reflection of love, labor, and land.
Inserted sentence: Songs of the Soil: Stories woven from the threads of farm life. Each lyric a reflection of love, labor, and land.
read sentence: Island Heartbeat: Feel the rhythm of the sands, the melody of the ocean. Music that carries the soul to the shores of tranquility.
Inserted sentence: Island Heartbeat: Feel the rhythm of the sands, the melody of the ocean. Music that carries the soul to the shores of tranquility.
read sentence: Voices of the Opera: Power and passion collide in soaring melodies. Experience the dramatic tales as told by the masters of vocal art.
Inserted sentence: Voices of the Opera: Power and passion collide in soaring melodies. Experience the dramatic tales as told by the masters of vocal art.
read sentence: Disco Days, Disco Nights: When the lights spun as fast as the records. Dive back into the era where the beats dictated the pace of the heartbeat.
```

```
Inserted sentence: Disco Days, Disco Nights: When the lights spun as fast as the records. Dive back into the era where the beats dictated the pace of the heartbeat.
read sentence: Punk Spirit: Not just music, but a movement. The raw energy and the untamed melodies that shaped a generation ready to stand.
Inserted sentence: Punk Spirit: Not just music, but a movement. The raw energy and the untamed melodies that shaped a generation ready to stand.
read sentence: Metal Titans: Heavy riffs and thunderous drums echo through the halls of rock. Relive the power that forged the metal legacy.
Inserted sentence: Metal Titans: Heavy riffs and thunderous drums echo through the halls of rock. Relive the power that forged the metal legacy.
read sentence: Epic Orchestras: Tales told in the language of strings and horns. Each composition a chapter, every note a word in the saga of sound.
Inserted sentence: Epic Orchestras: Tales told in the language of strings and horns. Each composition a chapter, every note a word in the saga of sound.
read sentence: Rhymes of the Street: The beat of the concrete, the rhythm of the city. Hear the stories of the streets narrated by the voices of the poets.
Inserted sentence: Rhymes of the Street: The beat of the concrete, the rhythm of the city. Hear the stories of the streets narrated by the voices of the poets.
read sentence: Evening Jazz: Let the troubles of the day melt away with each note. A smooth transition from day to night through the calming essence of jazz.
Inserted sentence: Evening Jazz: Let the troubles of the day melt away with each note. A smooth transition from day to night through the calming essence of jazz.
read sentence: Nights of Soul: Where the heart sings the blues, and the soul feels the beats. Dive deep into the emotional landscapes painted by soulful melodies.
Inserted sentence: Nights of Soul: Where the heart sings the blues, and the soul feels the beats. Dive deep into the emotional landscapes painted by soulful melodies.
Cassandra connection closed
Finished loading data
```

If you get no error, it means the articles were vectorized and loaded into the collection.

Task 6: Create a service endpoint

In this section, you will create a service with an endpoint that can accept a sentence as input and return a song with the closest meaning as a response. Let's see this in action using the curl command that looks for songs about nations.

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" \
-d '{"text": "song about nations"}' | jq .
```

Result:

```
{
  "title": "Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.",
  "similarity": 0.7019355893135071
},
{
  "title": "Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.",
  "similarity": 0.7019355893135071
},
{
  "title": "Island Heartbeat: Feel the rhythm of the sands, the melody of the ocean. Music that carries the soul to the shores of tranquility.",
  "similarity": 0.6959856748580933
}
}
```

As you can see, the endpoint returns songs about borders, shores, and islands. While the titles don't use the word nation explicitly, the results have that notion somewhere in the context.

Let's create this service from scratch now.

Create files and import utilities

Create a file called `server.js`

```
touch server.js
```

Add the following code to the file:

```
const express = require('express');
const { vectorizeText, connectToCassandra, closeCassandraConnection, CASSANDRA_KEYSPACE, CASSANDRA_TABLE } = require('./util');
const app = express();
app.use(express.json());
app.post('/search', async (req, res) => {
  const text = req.body.text;
  if (!text) {
    return res.status(400).send({ error: 'Text is required' });
  }
  try {
    // use the utility function to connect to the database.
    const client = await connectToCassandra();
    // create vectorize the database using the utility code.
    const queryVector = await vectorizeText(text);
    // create the query for the database using the embedding column to search using the cosine similarity.
    // we are ordering by similarity and only asking for the first three results.
    const query = `SELECT title, similarity_cosine(embedding, [${queryVector}]) as similarity FROM ${CASSANDRA_KEYSPACE}.${CASSANDRA_TABLE} ORDER BY embedding ANN OF [${queryVector}] LIMIT 3`;
    const result = await client.execute(query);
    if (result.rowLength > 0) {
      // map through all rows to construct a response array
      const matches = result.rows.map(row => ({
        title: row['title'],
        similarity: row['similarity']
      }));
      // return HTTP OK with the matches.
      res.status(200).send(matches);
    } else {
      // return 200 with message saying nothing was found.
      res.status(200).send({ message: 'No similar sentences found.' });
    }
  } catch (error) {
    console.error('Error:', error);
    // send back 500 if something goes wrong
    res.status(500).send({ error: 'Internal server error' });
  } finally {
    // finally, close the database connection using the method defined in the utility file.
    await closeCassandraConnection();
  }
});
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

These are a lot of codes. Let's go through it line by line.

Import required libraries

```
const express = require('express');
const { vectorizeText, connectToCassandra, closeCassandraConnection, CASSANDRA_KEYSPACE, CASSANDRA_TABLE } = require('./util');
```

As usual, the first step is to import external and internal libraries. In this case, you import express and all the awesome functions we defined in the util.js file.

```
app.post('/search', async (req, res) => {
  const text = req.body.text;
  if (!text) {
    return res.status(400).send({ error: 'Text is required' });
  }
}
```

Next, you create the POST endpoint and check that the user is sending some text with the request. If not, you return a status of 400, telling the client that a text is required.

```
// use the utility function to connect to the database.
const client = await connectToCassandra();
// create vectorize the database using the utility code.
const queryVector = await vectorizeText(text);
// create the query for the database using the embedding column to search using the cosine similarity.
// we are ordering by similarity and only asking for the first three results.
const query = `SELECT title, similarity_cosine(embedding,  [${queryVector}]) as similarity FROM ${CASSANDRA_KEYSPACE}.${CASSANDRA_TABLE} ORDER BY embedding ANN OF [${queryVector}] LIMIT 3`;
const result = await client.execute(query);
```

Next, connect with the database and vectorize the input text. Once you have the embedding, use the Cassandra DB query to check for cosine similarity using the `similarity_cosine` method. Store a similar number in the result and limit the result to 3.

```
if (result.rowLength > 0) {
  // map through all rows to construct a response array
  const matches = result.rows.map(row => ({
    title: row['title'],
    similarity: row['similarity']
  }));
  // return HTTP OK with the matches.
  res.status(200).send(matches);
} else {
  // return 200 with message saying nothing was found.
  res.status(200).send({ message: 'No similar sentences found.' });
}
```

Finally, if there are any matches, return a status of HTTP 200 along with the three matches. Otherwise, you return the message `No similar sentences found..`

Start the server

The last part of the code starts the app server and listens on port 3000.

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Run the server

You can use the same terminal you used for the other tasks or create a new terminal. To run the server, use the `node` command.

```
node server.js
```

You should see an output as follows:

```
node server.js
```

```
Platform node has already been set. Overwriting the platform with node.
Server is running on http://localhost:3000
Connected to Cassandra
Cassandra connection closed
```

If you execute the curl command shown at the beginning of this lab, you should see the song titles related to the nation through the words islands, borders, and shores.

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" \
-d '{"text": "song about nations"}'
```

Output:

```
[{"title": "Unity in Rhythm: Songs that traveled beyond borders to bring us together. Celebrating diversity through the universal language of music.", "similarity": 0.7019355893135071}, {"title": "Unity in Rhythm: Songs that traveled beyond borders to bring us together. Ce
```

You can try other examples as well. Here is one more with the text "weather":

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" \
-d '{"text": "song about the weather"}' | jq .
% Total    % Received % Xferd  Average Speed   Time    Time     Current
  Dload  Upload   Total             Spent    Left   Speed
100    570    100    536    100    34      2520    159   --:--:--   --:--:--   --:--:--   2766
[
  {
    "title": "Prairie Songs: Sung under open skies, carried by the breeze. Tales of heartland, sung with soul, echoing the spirit of the plains.",
    "similarity": 0.7270854711532593
  },
  {
    "title": "Prairie Songs: Sung under open skies, carried by the breeze. Tales of heartland, sung with soul, echoing the spirit of the plains.",
    "similarity": 0.7270854711532593
  },
  {
    "title": "Rhymes of the Street: The beat of the concrete, the rhythm of the city. Hear the stories of the streets narrated by the voices of the poets.",
    "similarity": 0.7021987438201904
  }
]
```

Again, you are using the tool jq to format the output so it's easier to read.

Congratulations! You successfully installed the Cassandra DB 5.0 database in Docker with vector features, created vector embeddings for song titles, created an index on this vector, and finally created an endpoint to retrieve songs similar to user input.

Authors

UL